杭州电子科技大学

计算机学院

编译原理课程实践

项目 1　预备实验

预备实验 任务书

(课程代码：S0512080)

课程名称	编译原理课程实践
课程代码	S0512080
学时（建议）	课内 3 学时；课外完成配置与调研
文档版本	v2.5
文档日期	2026-10
编制	杭州电子科技大学 计算机学院 编译原理课程组
适用专业	计算机科学与技术、计算机科学英才班、智能计算与数据科学、计算机科学与技术（第二学士学位）

杭州电子科技大学计算机学院

2026 年 10 月

一、项目概述

1.1 项目基本信息

表 1　项目基本信息

项目	内容
项目名称	预备实验
项目类型	验证（对应大纲表 2）
学 时	课内 3 学时用于指导与验收；课外补齐配置、练习与调研
教学方式	课堂讲授 + 上机实操 + 课外调研
适用专业	计算机科学与技术、计算机科学英才班、智能计算与数据科学、计算机科学与技术（第二学士学位）
对应课程目标	目标 1（设计/开发解决方案）、目标 4（使用现代工具）

1.2 项目目标

认识编译工具链四阶段及其输入输出，区分词法、语法和语义分析等内部阶段；使用 GCC、目标文件检查工具、GDB/LLDB 与 Flex；依据官方资料认识国产编译技术。

1.3 思政融入点

- 任务 1.0：科技报国（华为方舟/毕昇/OpenHarmony 编译器等）
- 任务 1.2：精益求精（结合“分阶段观察”展开）
- 任务 1.3：辩证思维（性能/可调试性/可移植性的权衡）
- 任务 1.4：创新思维（从工具使用者到工具思考者）
- 任务 1.5：科技报国（国产编译器调研）

二、任务清单

本项目包含 6 个任务（1 个环境配置 + 4 个课内任务 + 1 个课外调研）：

表 2　任务安排

任务号	任务名称	是否必做	建议学时	对应课程目标
1.0	环境配置（课前完成）	必做	课前完成	目标 4
1.1	走近 GCC：从源代码到可执行文件	必做	1 学时	目标 1, 4
1.2	阶段产物观察	必做	1 学时	目标 1, 4
1.3	GCC 编译选项实验：性能与诊断的取舍	必做	0.5 学时	目标 4
1.4	Lex 工具入门：编写与测试简单的词法分析器	必做	0.5 学时	目标 1, 2, 4
1.5	国产编译器调研（课外完成）	必做	随项目报告提交	目标 1, 4

课内 3 学时用于指导与验收；课前完成配置，课外补齐练习和调研，上机结束后 1 周内提交报告。

- 工具示例用于熟悉操作；数字分类和关键字与标识符识别由学生完成规则。提交时说明自己完成的内容，并能在验收中解释命令、错误原因与识别结果。
- 语言安排：本项目以 C 程序和 Flex 工具链为操作对象，用于观察编译、汇编、链接与生成式词法分析。后续算法项目允许自主选择实现语言；选择 Python 等语言的学生也需完成本项目的 C/Flex 工具操作。

三、各任务详情

任务 1.0 环境配置（课前完成）

教学目标

- 提前完成课程实践所需的全部工具安装，避免上机时挤占操作时间。
- 熟悉 Linux、macOS 和 Windows 三种平台的环境差异。
- 通过自检脚本验证环境可用性。

工具清单（必装）

表 3　工具清单

工具	用途	对应任务
GCC 驱动及编译/汇编/链接工具	编译、运行 C 程序	1.1-1.3
nm/objdump 与 ldd（Linux）；otool（macOS）	观察编译产物	1.2
GDB（Linux）/ LLDB（macOS）	调试	1.3
Flex	词法分析工具	1.4
C 标准库头文件	编译 C 程序	1.1-1.4

平台安装指引（详见实验指南）

- Ubuntu/Debian（首选）：apt install build-essential flex gdb git
- macOS：Xcode Command Line Tools 提供 Apple Clang、nm、otool 和 LLDB；另安装 Flex。观察 GNU GCC 内部工具时使用 Linux/WSL。
- Windows：使用 WSL 2 的 Linux 工具链；Git Bash 本身不能代替完整编译环境。

特别说明：先记录 gcc --version，macOS 的 gcc 通常实际调用 Apple Clang。本指南的 Flex 示例使用 %option noyywrap，无需统一指定 -lfl 或 -ll。

自检脚本（必跑）

详见实验指南任务 1.0：验证 C 编译运行、符号和依赖检查、调试器加载，以及 Flex 生成程序的实际识别结果。

```
$ gcc –Wall –Wextra –O0 –g check.c –o check
$ ./check
$ nm check | grep –E "main|add"
# 依赖与调试器检查按指南任务 1.0 的平台分支执行。
$ flex demo.l && gcc –o demo lex.yy.c
$ printf "123 hello 456\n" | ./demo
# check.c/demo.l 完整源码见指南；无需 –lfl/–ll。
```

提交要求

- 课前群内打卡“环境配置完成 + 截图”；报告第 1 章，简要说明平台、架构与工具版本，不另交环境报告。

任务 1.1 走近 GCC：从源代码到可执行文件

教学目标

- 认识 GCC 工具链的总体构成与各阶段分工。
- 掌握 C 程序从源代码到可执行文件的最简流程。
- 理解 .c / .i / .s / .o / 可执行文件 五种文件类型的本质。
- 理解编译单元翻译与链接的分工：链接涉及符号解析、重定位和目标文件/库的组成。

GCC 是工具链驱动程序；-E/-S/-c 决定停止阶段，-o 只指定输出文件名。词法、语法、语义分析、IR、优化和代码生成属于编译器内部工作。

操作要点

（1）用编辑器创建 hello.c，输入下方 C 源代码；也可按指南中的完整终端命令创建文件。

```
#include <stdio.h>
#define GREETING "Hello, Compiler!"
int add(int a, int b) { return a + b; }
int main(void) {
    int x = 2, y = 3;
    printf("%s %d + %d = %d\n", GREETING, x, y, add(x, y));
    return 0;
}
```

（2）使用 gcc hello.c -o hello 编译，运行 ./hello；

（3）用 file hello 验证可执行文件类型；

（4）故意制造语法错误，观察 GCC 错误信息；

（5）拆分多文件 add.c + main.c，理解链接错误。

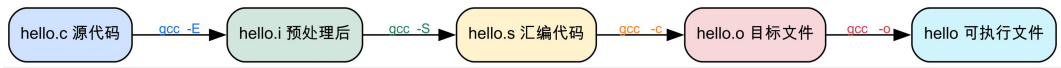


图 1　GCC 编译流水线

提交要求

- 3 个验收点：①Hello World 正确输出；②能读懂 GCC 错误信息；③能正确编译多文件项目
- 源文件 hello.c、bug.c、bug_fixed.c、add.c、main.c

任务 1.2 阶段产物观察

教学目标

- 深入理解 四阶段（预处理、编译、汇编、链接）的输入/输出。
- 掌握 GCC 各阶段对应选项（-E / -S / -c）。
- 通过对比源代码、汇编代码和可执行文件，理解各阶段的作用。

四阶段工具链

表 4　工具链阶段与产物

阶段	工具	输入	输出	GCC 选项
预处理	cpp	.c + 头文件	.i（预处理 C 文本，可含行标记）	-E
编译	cc1	.i	.s（汇编代码）	-S
汇编	as	.s	.o（目标文件）	-c
链接	ld	目标文件与库	可执行文件	不加 -E/-S/-c；-o 指定名称

操作要点

（1）gcc -E hello.c -o hello.i，观察宏和头文件展开，记录本机行数；以 # 开头的行标记仍可能保留。

（2）gcc -S hello.i -o hello.s，寻找 main: 标签（macOS 通常为 _main:）。

（3）gcc -c hello.s -o hello.o，用 file / nm / objdump 观察；

（4）gcc hello.o -o hello_from_o，观察链接中的符号解析与目标文件组合；

（5）gcc -### hello.c 只打印计划调用的命令，不执行；内部工具名和临时文件随 GCC/Clang 实现变化。

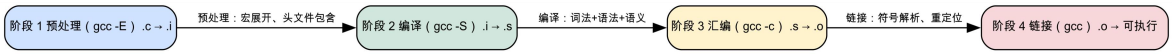


图 2　四阶段工具链

提交要求

- 验收：能解释四阶段的输入输出；观察 .c/.i/.s/.o/可执行文件五种文件；正确解释 -### 的作用。
- 报告含 四阶段命令、产物截图、关键观察

任务 1.3 GCC 编译选项实验：性能与诊断的取舍

教学目标

- 熟悉 GCC 常用选项（优化、警告、调试、预处理宏）。
- 比较不同编译选项对文件大小、汇编代码和运行时间的影响。
- 能够根据调试和运行需求选择编译选项。

实验 1：优化等级对比（必做）

```
$ cat > loop.c << 'EOF'
#include <stdio.h>
#include <time.h>
#define N 10000000LL
int main(void) {
    long long sum = 0;
    clock_t start = clock();
    for (long long i = 0; i < N; i++)
        sum += i * 2 + 1;
    clock_t end = clock();
    if (start == (clock_t)-1 || end == (clock_t)-1) {
        fputs("CPU time unavailable\n", stderr);
        return 1;
    }
    printf("sum = %lld\n", sum);
    printf("loop CPU time = %.6f s\n",
           (double)(end - start) / CLOCKS_PER_SEC);
    return 0;
}
EOF
$ gcc -O0 loop.c -o loop_O0
$ gcc -O1 loop.c -o loop_O1
$ gcc -O2 loop.c -o loop_O2
$ gcc -O3 loop.c -o loop_O3
$ gcc -O0 -S loop.c -o loop_O0.s
$ gcc -O1 -S loop.c -o loop_O1.s
$ gcc -O2 -S loop.c -o loop_O2.s
$ gcc -O3 -S loop.c -o loop_O3.s
$ wc -c loop_O0 loop_O1 loop_O2 loop_O3
$ wc -l loop_O0.s loop_O1.s loop_O2.s loop_O3.s
$ time ./loop_O0
$ time ./loop_O3
# 各版本 sum 均为 100000000000000；CPU 时间以本机实测为准
```

正确结果为 N^2=100000000000000。程序用 C 标准库 clock() 记录循环区间消耗的 CPU 时间，以秒输出 loop CPU time；不包含后续 printf 的时间，也不是墙钟时间。time ./loop_O0 等终端命令可另行观察整个进程的 real/user/sys。每个优化版本至少运行 5 次，分别记录循环 CPU 时间的中位数；0 或接近计时分辨率的结果应标注“无法可靠区分”。循环可能被编译期化简，不要为了计时擅自加入 volatile 改变本实验的优化条件。

对比表（手填）：

表 5　优化等级比较记录表

选项	文件大小	汇编行数	循环 CPU 时间中位数/秒（至少 5 次）
-O0			
-O1			
-O2			
-O3			

实验 2：警告与错误（必做）

使用 -O1 -Wall -Wextra 识别至少 5 类诊断，再使用 -Werror 观察警告升级为错误；按实际类别记录并修复。故意含未定义行为的代码仅用于编译诊断，不运行。示例见指南。

实验 3：调试信息（必做）

对比 -O0 与 -O0 -g 的源代码行及局部变量调试信息。无 -g 不等于完全不能调试；macOS 可用 LLDB。

实验 4：预处理宏（必做）

用 -DDEBUG 控制示例 test.c 的日志代码；DEBUG 宏与产生调试信息的 -g 不等价。完整 test.c 见指南。

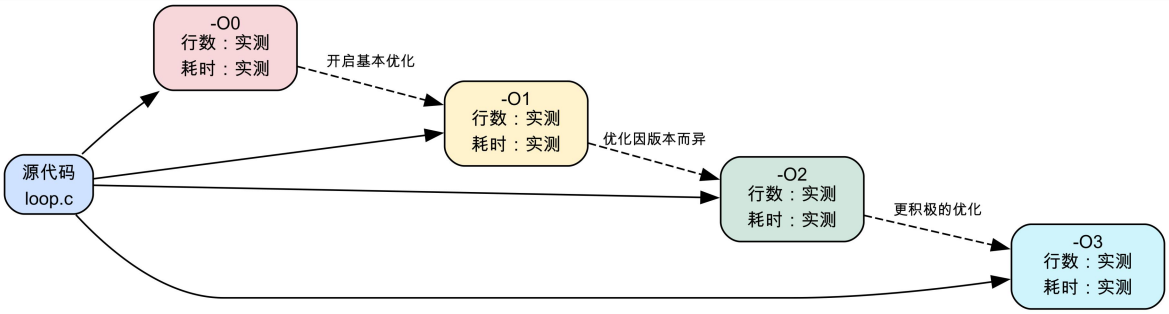


图 3　优化等级对比

提交要求

- 4 个实验的实际记录、优化对比表及结论依据；截图可汇总

任务 1.4 Lex 工具入门：编写与测试简单的词法分析器

教学目标

- 了解 Lex/Flex 工具的“三段式”结构与工作流。
- 亲手编写并运行最简单的 .l 文件。
- 了解正规式如何通过自动机构造用于词法分析。
- 为项目 2 任务 2.1 的 Thompson 构造学习做准备，认识工具背后的自动机算法。

实验 1：最简整数识别器

```
$ cat > demo.l << 'EOF'
%option noyywrap
%{
#include <stdio.h>
%}
%%
[0-9]+       { printf("INT: %s\n", yytext); }
[ \t\r\n]+  { /* skip whitespace */ }
.           { /* skip other characters */ }
%%
int main(void) { yylex(); return 0; }
EOF
$ flex demo.l
$ gcc -o demo lex.yy.c
$ printf '123 hello 456\n' | ./demo
# 预期：分两行输出 INT: 123 和 INT: 456
```

实验 2：多类词法单元识别——以数字字面量为例

在 numbers.l 框架中编写多条词法规则，识别十进制、八进制、十六进制整数和简单浮点数，分别输出 DEC、OCT、HEX、FLOAT 及匹配文本。简单浮点数限定为小数点两侧均有数字的形式，不含指数部分；具体格式见实验指南。

DEC、OCT、HEX 是区分整数表示形式的教学标记，并非要求实际语言必须将不同进制的整数定义为不同 token 类别。完成给定样例后，自选至少两组边界输入，先预测切分结果，再运行核对并解释原因。本实验不要求完整诊断非法数字；负号不属于数字规则。

实验 3：小型 C 风格词法识别器（关键字、标识符和整数）

在 mini_c.l 框架中编写 int、if、while 关键字、标识符和十进制整数规则，分别输出 KW、ID、INT 及匹配文本。标识符以英文字母或下划线开头，后接零个或多个英文字母、数字或下划线；整数规则采用 [0-9]+。按输入顺序连续识别，跳过空白，保留对未处理字符的提示。

使用 int、integer、ifx、while2 等输入，先预测分类再运行核对。交换关键字规则与标识符规则的顺序，记录修改前后的结果，解释最长匹配与等长规则优先级。运算符、分隔符和注释识别作为扩展练习，不列入必做要求。

实验 1 的 demo.l 提供完整工具示例；实验 2 和实验 3 的识别规则由学生自行完成。三个实验的操作步骤、构建命令与测试示例见实验指南。

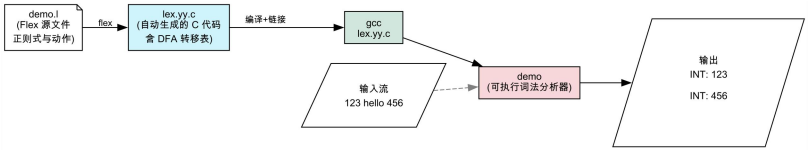


图 4　Flex 工具链

提交要求

- demo.l 运行记录；完成的 numbers.l 与 mini_c.l 源码及预期与实际输出；至少两组数字边界输入的预测、结果和解释。
- 至少一组模式或规则顺序修改前后的对比及原因解释

- 解释最长匹配与等长规则优先级，给出一个自测反例；理解正规式用于描述词法规则、Flex 借助自动机完成识别，具体构造算法在项目 2 中学习。

任务 1.5 国产编译器调研（课外完成）

教学目标

- 了解国产编译器的发展现状与典型代表。
- 体会编译器作为基础软件对国家信息安全的战略意义。
- 树立科技报国的使命感。

选题清单（6 选 1 或自选）

表 6　调研选题

选题	代表	主页
A	华为方舟编译器（Ark Compiler）	gitee.com/openarkcompiler/openarkcompiler
B	华为毕昇编译器（BiSheng）	gitee.com/openeuler/bishengcompiler
C	OpenHarmony 编译器体系	gitee.com/openharmony
D	龙芯 LoongArch 工具链	gitee.com/loongson
E	阿里云 / 字节跳动自研编译器	（内源项目）
F	自选	经教师确认

报告模板（800 字以上）

调研对象：所选编译器或工具链名称、版本及定位。

关键发现：支持语言、目标架构、工具链组成及一项有来源支持的技术特征。

我的思考：联系本项目，说明适用场景、局限或一次可复现体验。

参考文献：至少 2 条，至少 1 条为官方仓库、文档或白皮书；标明链接及访问日期。

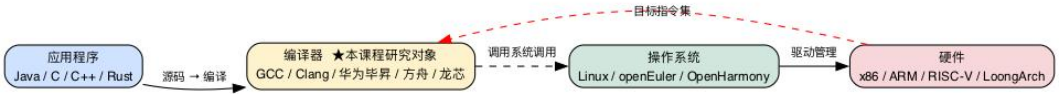


图 5　基础软件栈

提交要求

- 800 字以上调研报告，附在项目报告“思政融合”章节，与项目报告一同提交。

四、考核要点

4.1 成绩占比

本项目占总评成绩 5%（基础实验“预备实验”项）。

4.2 主要扣分情形

验收结合源文件、实际测试、错误修复、阶段产物解释及现场操作。只运行给定示例不能替代自行完成的识别规则。

- 任务 1.2 缺少阶段产物记录或不能解释阶段输入输出
- 任务 1.3 优化对比表数据空白
- 任务 1.4 未完成要求的识别规则，或不能结合测试解释规则的作用
- 任务 1.5 思政融合章节缺失或照抄网络内容

4.3 加分项

- 完成必做任务后，可选择比较更多编译选项并说明适用前提，探索 Flex 的开始条件等特性，或结合论文、白皮书深化调研。

AI 辅助编写说明

本次课程材料使用了多个 AI 辅助工具，协助开展实验任务设计、实验指南编写、示例整理以及文字和排版润色。材料由编写者统筹取舍，课程组负责审核与定稿。